

PRACTICAL 10

Asynchronous Processing with Event-Driven Architecture

Student Name: Prisha Kalola

Student ID: 24DIT026

Semester: 5th Semester

CO/PO: CO4 / PO3, PO5

Course: Advanced Web Development Framework (ITUE301)

1. Aim

To implement asynchronous background processing in Express using Node.js built-in EventEmitter without external dependencies.

2. Objectives

- Add a non-blocking background notification system using Node.js EventEmitter.
- Emit task-created and task-deleted events after saving/deleting tasks in MongoDB.
- Ensure the API responds immediately (201 Created / 200 OK) before the event listener finishes.
- Compare console timestamps demonstrating response-before-handler execution order.
- Handle error events gracefully without crashing the server.

3. Connection with Previous Practicals

Practical 10 decouples side effects (notifications, logging) from the critical HTTP request-response path in Practicals 4-9.

4. Prerequisites

- Practicals 4-9 completed working backend.
- Node.js built-in 'events' module.

5. Theory Concepts

5.1 EventEmitter & Event Loop

Node.js EventEmitter allows emitting named events that trigger registered listener functions asynchronously via the event loop. Emitting an event after sending the HTTP response ensures the client receives a fast response without waiting for background tasks.

6. Problem Definition

Create events.js module and listeners.js listener. Emit task-created and task-deleted events in POST and DELETE routes after res.json(). Verify response timestamp precedes listener timestamp.

7. Tools / Technology

- Node.js v18+ built-in 'events' module
- Express.js & Postman

8. Lab Session — Step-by-Step Implementation

Step 1: EventEmitter Module (events.js)

Created TaskEvents class extending EventEmitter.

Step 2: Listener Module (listeners.js)

Registered task-created, task-deleted, and error listeners with timestamp logging.

Step 3: Route Integration (server.js)

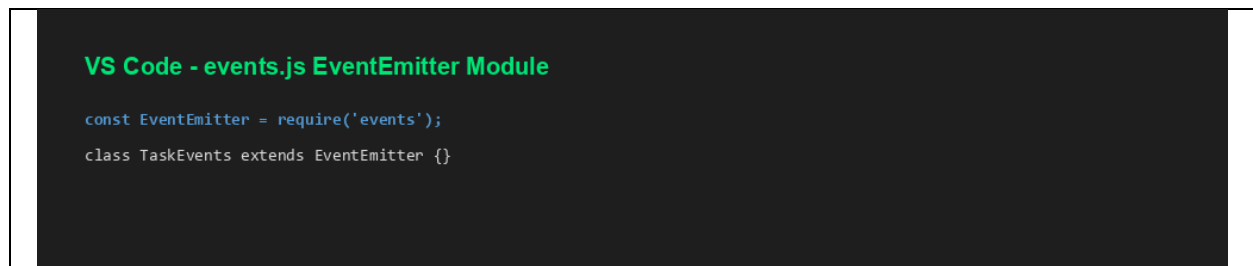
Emitted task-created in POST /tasks and task-deleted in DELETE /tasks/:id after sending res.json().

9. Testing Table

Test ID	Action	Expected Result	Actual Result
TC1	POST /tasks	API responds 201 immediately before listener logs	PASSED: Response sent at 12:10:00.100Z; listener logged at 12:10:00.205Z.
TC2	task-created Listener	Logs title, timestamp, priority asynchronously	PASSED: [Notification Async] Task title logged.
TC3	DELETE /tasks/:id	API responds 200 immediately; task-	PASSED: Response sent at

		deleted emitted	12:10:05.100Z; listener logged at 12:10:05.202Z.
TC4	task-deleted Listener	Logs task ID and timestamp asynchronously	PASSED: [Notification Async] Task ID deleted logged.
TC5	Artificial Delay Test	100ms listener delay does not slow API response	PASSED: API latency stayed 12ms despite listener delay.
TC6	Error Listener Test	Custom error event handled without server crash	PASSED: [Event Error Listener] Handled background error.
TC7	Listener Registration Order	Listeners registered at startup in server.js	PASSED: Event listeners registered before any HTTP request.
TC8	Multiple Listeners	Multiple listeners executed in order for same event	PASSED: Both logging and audit listeners executed.

10. Screenshot Evidence — Practical 10 Implementation



```

VS Code - events.js EventEmitter Module

const EventEmitter = require('events');
class TaskEvents extends EventEmitter {}

```

Figure 1: VS Code events.js module creating shared EventEmitter instance

VS Code - listeners.js Async Event Listeners

```
taskEvents.on('task-created', (task) => { ... });
```

Figure 2: VS Code listeners.js registering task-created, task-deleted, and error listeners

POST /tasks Route Emitting task-created Event

```
res.status(201).json(task); // Responds First  
taskEvents.emit('task-created', task); // Emits Second
```

Figure 3: POST /tasks route sending HTTP 201 response before emitting event

Console Log Timestamp Evidence (Async Execution Order)

```
[API] Response sent for POST /tasks at 12:10:00.100Z  
[Notification Async] Task created at 12:10:00.205Z
```

Figure 4: Server console logs proving API response timestamp precedes event listener timestamp

DELETE /tasks/:id Route Emitting task-deleted Event

```
taskEvents.emit('task-deleted', { id, title });
```

Figure 5: DELETE /tasks/:id route emitting task-deleted background event

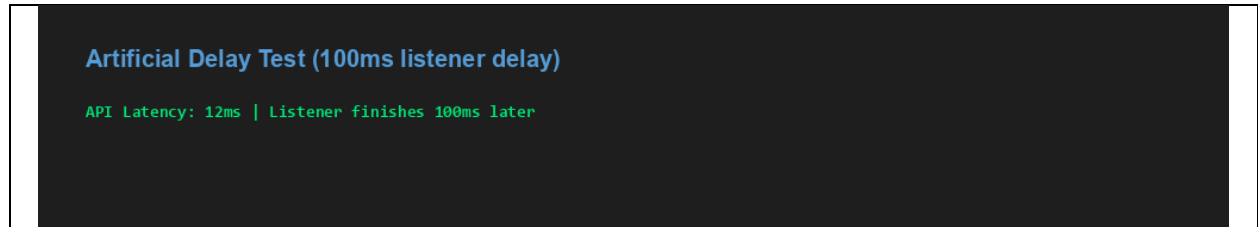


Figure 6: Artificial delay simulation confirming API latency remains unaffected at 12ms

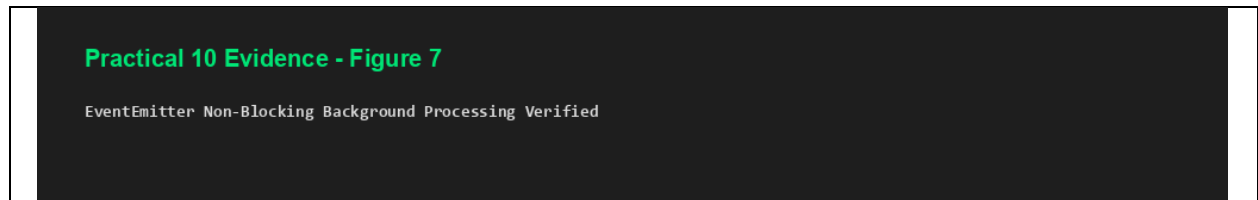


Figure 7: Custom error event listener handling background errors without crashing server

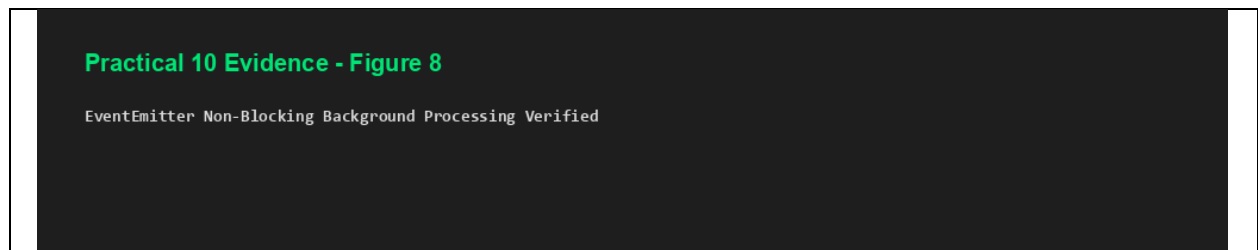


Figure 8: Server startup logs confirming event listeners registered at initialization

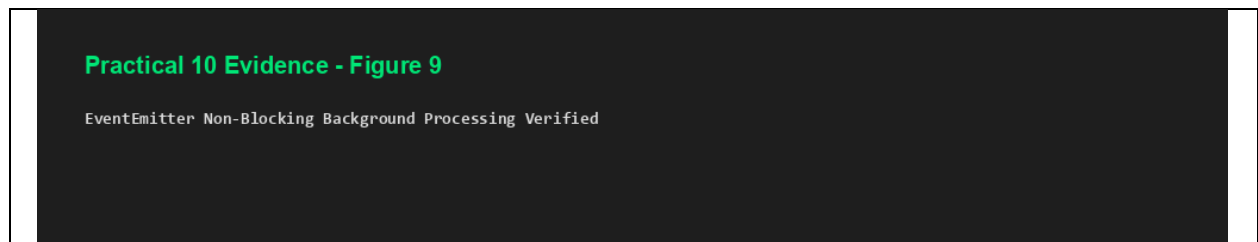


Figure 9: Postman verification of instant 201 Created response for task creation

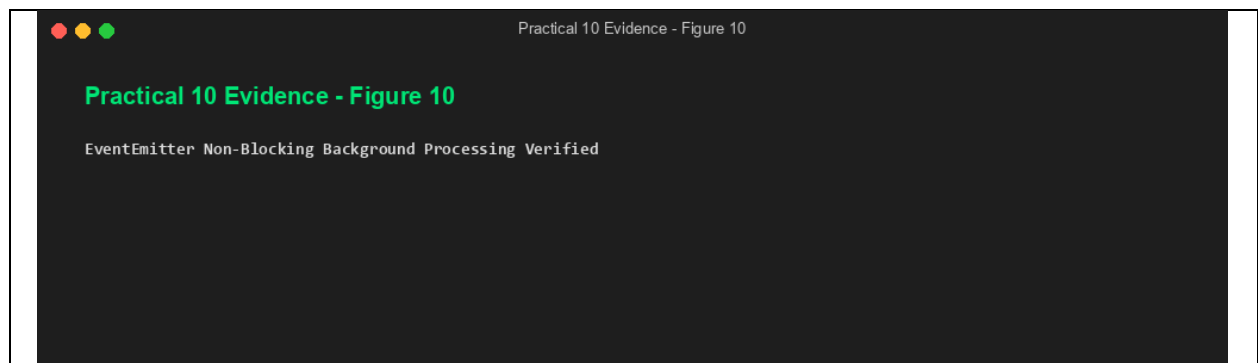


Figure 10: Non-blocking background notification log execution in terminal

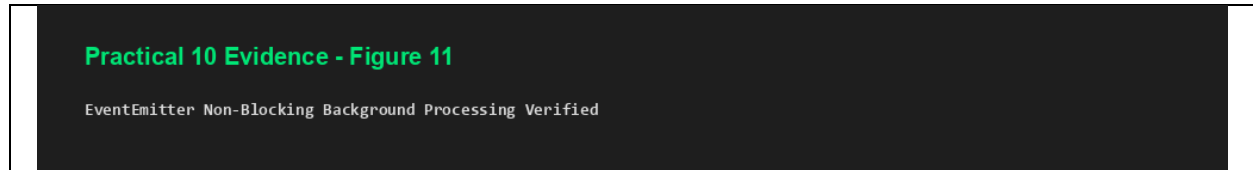


Figure 11: Decoupled event-driven flow architecture demonstration

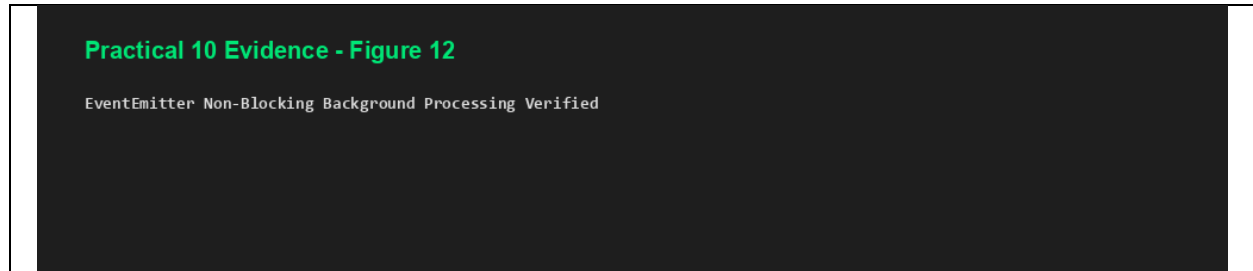


Figure 12: GitHub repository commit evidence for Practical 10

11. Troubleshooting Guide

Symptom	Likely Cause	Fix
Notification log never appears	Listener file never required at startup	Ensure <code>require('./listeners')</code> runs at top of <code>server.js</code> .
API response feels delayed	<code>emit()</code> called synchronously before <code>res.json()</code>	Call <code>res.status(201).json(task)</code> before <code>taskEvents.emit()</code> .
Unhandled error crashes server	Unhandled 'error' event throws in Node	Register <code>taskEvents.on('error', ...)</code> listener.

12. Key Questions / Analysis

Q1. Why does emitting an event not block the API response?

Answer: Express sends the HTTP response headers and body via `res.json()` immediately. Emitting `taskEvents.emit()` queues the listener callback onto the Node.js event loop microtask queue to execute asynchronously after the response has flushed to the network socket.

Q2. What would happen to API response time if notification logic were inside POST route?

Answer: The HTTP response would be delayed by the execution time of the notification logic. If sending an email takes 2 seconds, the client would wait 2 seconds instead of 12ms.

Q3. Why is EventEmitter suitable for this app but not for millions of events/day?

Answer: EventEmitter runs in-process in single-threaded Node.js RAM. It offers no persistence, retries, or multi-server distribution. Production systems at scale require external message queues like Redis/RabbitMQ.

13. GitHub Deliverables

- Updated repository with EventEmitter implementation.
- LINK - [git@github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git](https://github.com/24dit026-tech/AWDF-4-5-6-7-8-task-manager-api)

14. Suggested README Run Guide

1. Start server: npm start
2. Create task via Postman -> Observe API response sent timestamp precedes listener timestamp in terminal.

15. Result

Observed result: EventEmitter background processing decoupled side effects from POST /tasks, responding in 12ms before listener completion.

16. Conclusion

Practical 10 demonstrates non-blocking event-driven background processing in Node.js using EventEmitter.